

实验报告

233020010046-张子鑫

1.构建地图

使用 SLAM Toolbox 构建地图：

```
$ export TURTLEBOT3_MODEL=burger
$ ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
新终端
$ export TURTLEBOT3_MODEL=burger
$ ros2 launch turtlebot3_cartographer cartographer.launch.py use_sim_time:=True
新终端
$ export TURTLEBOT3_MODEL=burger
$ ros2 run turtlebot3_teleop teleop_keyboard
```

通过键盘控制机器人运动构建地图，保存

2.自主导航

采用Navigation2让机器人自主巡航

3.多点巡航

设置目标点，让机器人读取目标节点，接受机器人到达的信息

3.1 patrol_node.py

基于 **Nav2 Simple Commander** 的 `BasicNavigator` 接口调用导航动作 (`NavigateToPose`)，读取 YAML 配置的多个航点 (waypoints)，按顺序发送目标点，支持失败重试、超时取消、到点驻留

- `declare_parameter()` 声明参数，使巡逻行为可配置，便于调参

`waypoints_yaml`：航点 YAML 文件路径

`frame_id`：目标点所在坐标系，默认map

`loop`：是否循环巡逻 (True/False)

`dwell_time`：到点后停留时间 (秒)

`goal_timeout_sec`：单目标导航超时时间 (秒)

`retry_per_goal`：单个航点失败重试次数

```
self.declare_parameter("waypoints_yaml", "")
self.declare_parameter("frame_id", "map")
self.declare_parameter("loop", True)
self.declare_parameter("dwell_time", 1.0)
self.declare_parameter("goal_timeout_sec", 120.0)
self.declare_parameter("retry_per_goal", 1)
```

- 航点读取

通过 `yaml.safe_load()` 读取, 并做空列表检查

```
data = yaml.safe_load(p.read_text(encoding="utf-8"))
wps = data.get("waypoints", [])
if not wps:
    raise RuntimeError("No waypoints found in YAML (expected key:
waypoints).")
```

- 目标位姿构造

Nav2 接收的目标为 `PoseStamped`, `yaw_to_quat()` 将平面 yaw 转换为 quaternion (roll/pitch 视为 0)

```
def yaw_to_quat(yaw: float):
    return (0.0, 0.0, math.sin(yaw / 2.0), math.cos(yaw / 2.0))
```

在 `make_pose()` 中填充目标位姿:

- `frame_id`: 使用 `map` (默认)
- `stamp`: 使用节点时钟 `now()` (与 `use_sim_time` 一致)
- `position`: 填入 `x, y`
- `orientation`: 由 yaw 转四元数得到

- 航点可视化

节点发布 `MarkerArray` 到话题 `patrol_waypoints`, 在 RViz 中显示巡逻点位置

每个航点用一个 `SPHERE` 表示, id 为航点序号, 统一命名空间 `patrol_waypoints`

- 单目标导航 `go_to_one()`

- `goToPose(pose)` 发送目标

循环检查 `isTaskComplete()`

设定超时 `goal_timeout_sec`, 若超时则 `cancelTask()` 并返回 `CANCELED`

完成后 `getResult()` 获取最终结果 (SUCCEEDED/FAILED/CANCELED)

```
self.navigators.goToPose(pose)
while rclpy.ok() and not self.navigators.isTaskComplete():
    if self.get_clock().now() - start > timeout:
        self.navigators.cancelTask()
        return TaskResult.CANCELED
    rclpy.spin_once(self, timeout_sec=0.1)
return self.navigators.getResult()
```

- 巡逻主循环

- `run_patrol()` 实现完整巡逻策略:
 - 外层 `while rclpy.ok()`: 用于循环巡逻
 - 内层遍历所有航点: 逐点导航
 - 每点支持 `retry_per_goal` 次重试
 - 到点或放弃后执行驻留 `dwel1_time`

```

for i, wp in enumerate(self.waypoints):
    for attempt in range(self.retry_per_goal + 1):
        result = self.go_to_one(goal)
        if result == TaskResult.SUCCEEDED:
            ok = True
            break

```

- 启动方式：定时器触发避免阻塞

```
self.create_timer(1.0, self.start_once)
```

3.2 patrol.launch.py

提供航点配置文件路径参数（`waypoints_yaml`）；向巡逻节点注入运行参数，方便调参

- 默认航点路径的构造方式

```

default_waypoints = PathJoinSubstitution([
    FindPackageShare('patrol_waypoint'),
    'config',
    'waypoints.yaml'
])

```

`FindPackageShare('patrol_waypoint')` 在安装空间中定位包的 share 目录

- 启动参数声明

```

DeclareLaunchArgument(
    'waypoints_yaml',
    default_value=default_waypoints,
    description='Path to waypoint YAML file'
)

```

- 节点启动与参数注入

- Launch 文件通过 `Node()` 启动巡逻节点，并将参数写入节点参数服务器
- `waypoints_yaml`：航点 YAML 文件路径（由 `LaunchConfiguration` 获取，可被命令行覆盖）

`frame_id='map'`：目标点所在坐标系

`loop=True`：完成一轮航点后继续下一轮，实现循环巡逻

`dwel_time=1.0`：到达航点后停留时间（秒），便于观察与演示

`goal_timeout_sec=300.0`：单个目标的最大执行时间，超时会取消任务并进入重试、跳过逻辑

`retry_per_goal=1`：单航点失败重试次数

```

Node(
    package='patrol_waypoint',
    executable='patrol_node',
    name='patrol_node',
    output='screen',

```

```
parameters=[{
    'waypoints_yaml': waypoints_yaml,
    'frame_id': 'map',
    'loop': True,
    'dwell_time': 1.0,
    'goal_timeout_sec': 300.0,
    'retry_per_goal': 1,
}]
)
```

实验结果

- 初始设置四个目标点，机器人依次到达每一个目标点

```
✓ waypoints:
- {x: 1.4, y: 1.1, yaw: 0.0}
- {x: 2.0, y: 2.4, yaw: 1.0}
- {x: 1.4, y: 0.0, yaw: 2.0}
- {x: 0.0, y: 0.0, yaw: 3.0}
```

